

# TP2 - App UI & Navigation

Mobile & Ubiquitous Computing 2026/2027  
Ciências, Departamento de Informática

Diogo Branco

# Building UIs & Navigation

#2 Class

# Evaluation

Assignment #1 - Favorite Travel  
Badge



# Goals

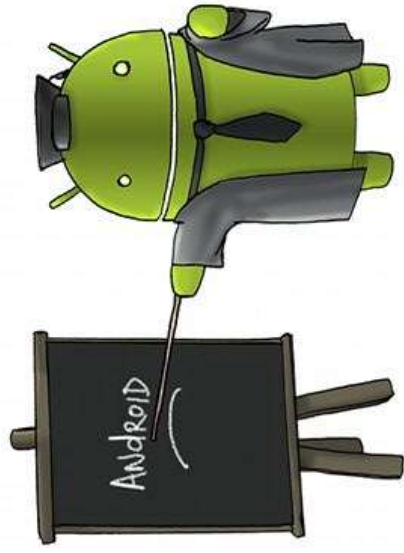
Handle user interactions

Refresh UI

Navigate between screens

Send simple data between screens

# 02



# UI Fundamentals

## Compose modifiers

Modifiers allow you to decorate or augment a composable. Modifiers let you do these sorts of things:

- Change the composable's size, layout, behavior, and appearance
- Add information, like accessibility labels
- Process user input
- Add high-level interactions, like making an element clickable, scrollable, draggable, or zoomable

Modifiers are standard Kotlin objects. Create a modifier by calling one of the `Modifier` class functions:

```
@Composable
private fun Greeting(name: String) {
    Column(modifier = Modifier.padding(24.dp)) {
        Text(text = "Hello,")
        Text(text = name)
    }
}
```

ModifierSnippets.kt 

<https://developer.android.com/jetpack/compose/modifiers>

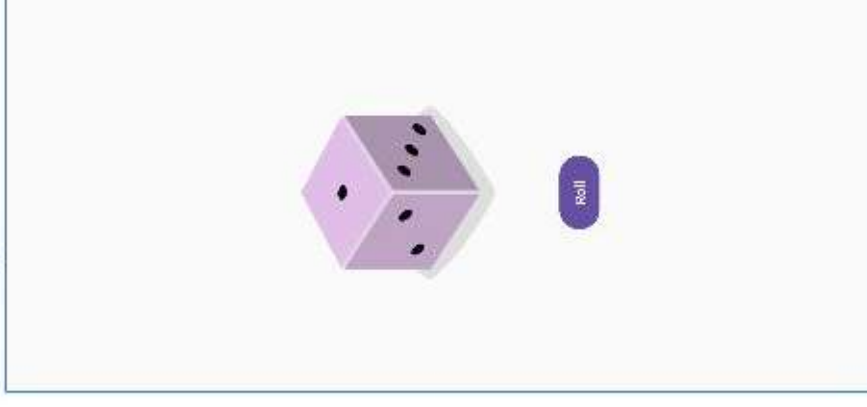
# Refreshing the UI

## Composables

- They can be used at any time to update the interface (recompose)
- They are stateless by default
- **Remember** keeps a value over time

## Example

var value by **remember** { mutableStateOf(1) }



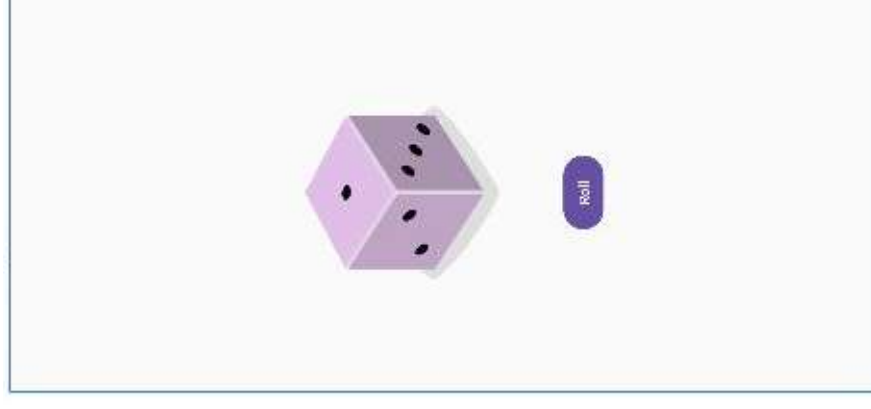
# Refreshing the UI

## *Reacting to changes in data*

- **`mutableStateOf()`**
  - Observable that triggers recompositions (UI updates)

## **Example**

var result by remember { **`mutableStateOf(1)`** }



# Tutorial Part 1 - Interactions and Updates

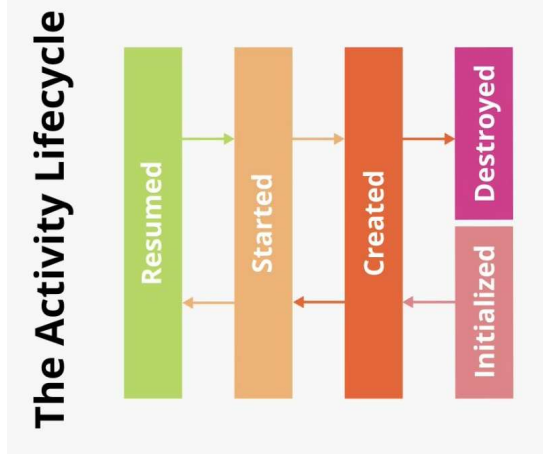
## Interactive Dice Roller

- Until step 7

<https://tinyurl.com/CMDiceRollerTP2>

# Tutorial Part 2 - Navigation

With current best practices, one activity might display multiple screens by swapping them in and out as needed



# Tutorial Part 2 - Navigation

1. Create a new kotlin file (not a class!) with a composable to show the dice result (DiceResult.kt)

```
@Composable
fun DiceResult(modifier: Modifier = Modifier) {
    var result by remember { mutableStateOf<Int> { value: 1 } }
    val imageResource = when (result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    Column(modifier = modifier, horizontalAlignment = Alignment.CenterHorizontally) { this: ColumnScope
        Image(painter = painterResource(imageResource), contentDescription = result.toString())
        Button(
            onClick = { /*TODO transition to Main*/ },
        ) { this: RowScope
            Text(text = (stringResource(R.string.back)), fontSize = 24.sp)
        }
    }
}
```

Add back to the string resources

# Tutorial Part 2 - Navigation

2A1. Add to build.gradle (app). 2.10.2 is likely the most recent version.

```
dependencies {  
    implementation(libs.androidx.navigation.compose)
```

2A2. Add to libs.versions.toml

```
[versions]  
navigationCompose = "2.10.2"  
  
[libraries]  
androidx-navigation-compose = { module =  
    "androidx.navigation:navigation-compose", version.ref =  
    "navigationCompose" }
```

2B. Add to build.gradle (app). (alternative)

```
dependencies {  
    implementation("androidx.navigation:navigation-compose:2.10.2")
```

# Tutorial Part 2 - Navigation

- **A route is a string that corresponds to a destination (similar to a URL)**
  - Example: `navController.navigate("profile")`
- **Serves as a unique identifier**
  - Example: "home", "profile"
- **Destinations are typically a single Composable or group of Composables**
  - Example:
    - `composable(route = "profile") { UserProfileScreen() // <- O Destino Composable }`

# Tutorial Part 2 - Navigation

## 3. Create a new sealed class Screens.kt under a new package “navigation”

```
package com.example.diceroller.navigation

sealed class Screens(val route: String) {
    object Roll : Screens("roll_screen")
    object DiceResult : Screens("result_screen")
}
```

# Tutorial Part 2 - Navigation

## Parts of the Navigation Component

The Navigation component has three main parts:

- **NavController:** Responsible for navigating between destinations—that is, the screens in your app.
- **NavGraph:** Maps composable destinations to navigate to.
- **NavHost:** Composable acting as a container for displaying the current destination of the NavGraph.

# Tutorial Part 2 - Navigation

## 4. Create a new kotlin file NavGraph.kt (in navigation package)

```
import androidx.compose.runtime.Composable
import androidx.navigation.NavHostController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.rememberNavController
import androidx.navigation.compose.composable
import com.example.diceroller.DiceResult
import com.example.diceroller.DiceWithButtonAndImage
```

```
@Composable
fun NavGraph (navController: NavHostController) {
    NavHost(
        navController = navController,
        startDestination = Screens.Roll.route)
    {
        composable(route = Screens.Roll.route) {
            DiceWithButtonAndImage(navController = navController)
        }
        composable(route = Screens.DiceResult.route) {
            DiceResult(navController = navController)
        }
    }
}
```

It will show an error until you add  
navController as a parameter (next slides)

# Tutorial Part 2 - Navigation

## 5. Alter MainActivity and add NavController

```
import com.example.diceroller.navigation.NavGraph

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(
            DiceRollerTheme {
                val NavController = rememberNavController()
                NavGraph(navController = NavController)
            }
        )
    }
}
```

*"This remember function always returns the same object and allows us to not lose this object if we make configuration changes(e.g. screen orientation)"*

# Tutorial Part 2 - Navigation

## 6. In MainActivity add NavController to the @composable

```
fun DiceWithButtonAndImage NavController: NavController, modifier: Modifier = Modifier) {
```

## 7. In DiceResult do the same, add NavController to the @composable

```
@Composable  
fun DiceResult (NavController: NavController, modifier: Modifier = Modifier) {
```

# Tutorial Part 2 - Navigation

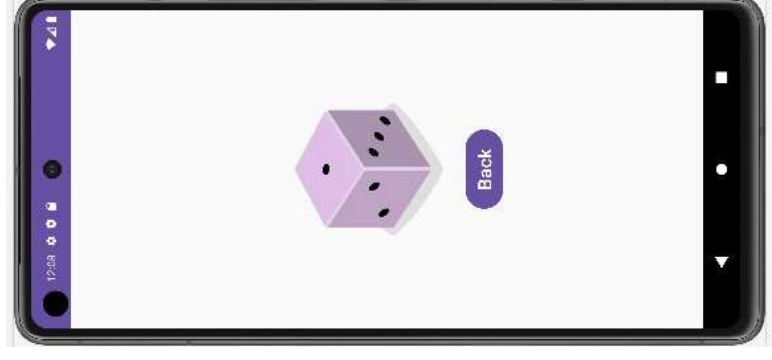
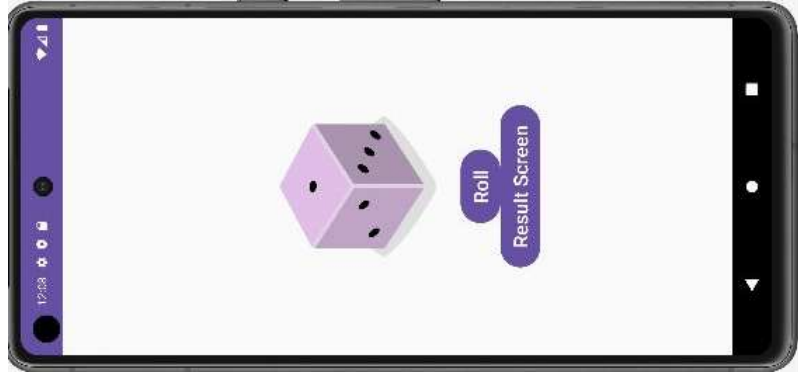
## 8. In MainActivity add a button to go to result

```
Button(  
    onClick = { navController.navigate(Screens.DiceResult.route) },  
) {  
    Text(text = stringResource(R.string.result_screen), fontSize = 24.sp)  
}
```

## 9. In DiceResult do the same, add a button to come back

```
Button(  
    onClick = { navController.navigate(Screens.Roll.route) },  
) {  
    Text(text = (stringResource(R.string.back)), fontSize = 24.sp)  
}
```

# Run!



*There will be interface differences, you can try and use the modifiers to align the interface as is here*

# Tutorial Part 3 - Sending Data

## 10. In Screens.kt add result variable

```
sealed class Screens(val route: String) {  
    object Roll : Screens("roll_screen")  
    object DiceResult : Screens("result_screen/{result}")  
}
```

## 11. In NavGraph.kt alter the route and handle the result

```
composable(route = Screens.DiceResult.route+ "?result={result}") { navBackStack ->  
    //extracting the argument  
    var resultShow: Int = navBackStack.arguments?.getString("result")?.toIntOrNull()?:1  
    DiceResult(navController, resultShow = resultShow)  
}
```

# Tutorial Part 3 - Sending Data

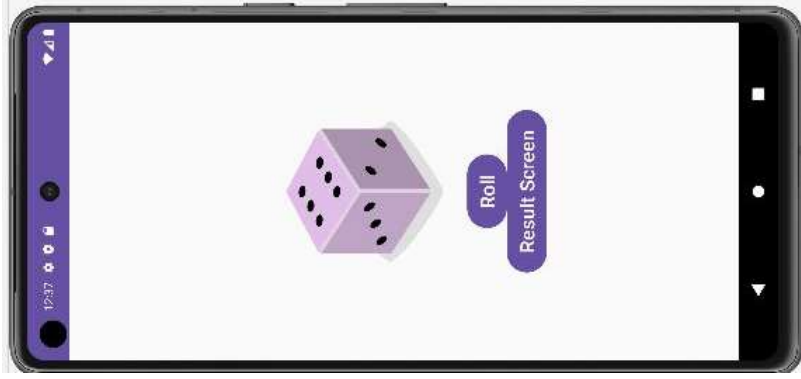
## 12. In MainActivity.kt change the onclick to navigate

```
onClick = {
    navController.navigate(
        Screens.DiceResult route //Just modify your route accordingly
    ).replace(
        oldValue = "{result}",
        newValue = result.toString()
    )
}
```

## 13. In DiceResult.kt retrieve and show it

```
fun DiceResult(navController: NavController, resultShow: Int = 1 , modifier: Modifier =
    Modifier.fillMaxSize().wrapContentSize(
        Alignment.Center)) {
    var result by remember { mutableStateOf(resultShow) }
```

# Run!



*Now it should pass the value and show the resulting die*

# Additional resources

## More Buttons Tutorial

<https://tinyurl.com/cmLemonadeMoreButtons>

## UI State

<https://developer.android.com/jetpack/compose/state>

## Navigation

<https://developer.android.com/jetpack/compose/navigation> (learn more)

<https://tinyurl.com/cmadditionalnavigation> (Pathway)

<https://dev.to/jbc7ag/jetpack-compose-navigation-tutorial-9en> (Another example)

# Other Additional Resources

## Debugger Tutorial

<https://tinyurl.com/CMAdditionalDebugger>

## Activity Lifecycles

<https://tinyurl.com/cmactivitylifecicle>

## Learn Lists

<https://tinyurl.com/cmlistspathway>

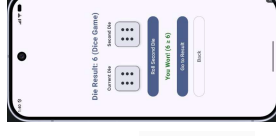
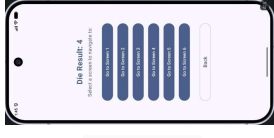
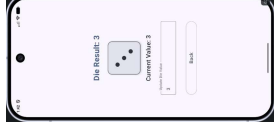
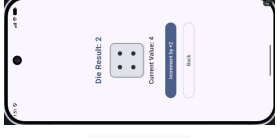
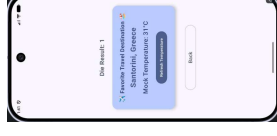
# Assignment #2 - Dice Screens

Create a new screen for each die result.

*The first screen must allow you to roll as much as you want, and transition only when selecting "Go to Result"*

Die results:

- 1 - Your Favorite Travel Badge (with a back button), and a "Refresh" button that randomly changes the mock temperature
- 2 - Die Result, back button, and button to increment die by two
- 3 - Die Result, TextField (that updates the die), back button
- 4 - Six button, one to go to each screen.
- 5 - Die Result and back button
- 6 - Current die result on one die, a button to roll a second die, and the result of that roll displayed on another die. After rolling, a message appears stating whether the user won (if the new roll is greater than or equal to the current one) or lost (if it is lower). If they win, a "Go to Result" button appears, allowing the user to navigate to the respective screen number of the new die (e.g., current roll - 2; new roll - 5; "You Won!") message appears, and clicking "Go to Result" shows one die with a 2 and the other with a 5).



# Assignment #2 - Dice Screens

